ATIVIDADE PRÁTICA **Aluno: Henzo Gabriel Volpe Cardoso**

RA: 26015432-2

Exercícios - Faça na ordem (esquenta → final)

Exercício 1 - $lookup - Enriquecendo dados (1,5 pontos) - FÁCIL

Contexto: O gerente de lore não sabe o que cada região significa e nem quando cada geração de Pokémon foi lançada. Enriqueça os dados!

a) (0,8) Com campeoes + regioes_info: Faça $lookup para trazer lider, caracteristica e cor da região de cada campeão. Mostre nome, regiao, lider, caracteristica. Ordene por regiao.
Resultado esperado: { nome: 'Garen', regiao: 'Demacia', lider: 'Garen', caracteristica: 'Justiça e honra' }

b) (0,7) Com pokemon + geracoes_info: Faça $lookup para trazer nome da geração, ano e lendário. Mostre name, generation, nomeGeracao, ano, lendario. Limite 10.
Resultado: { name: 'Bulbasaur', generation: 1, nomeGeracao: 'Kanto', ano: 1996, lendario: 'Mewtwo' }

Cole as 2 queries aqui:

Resposta:
a)

```
db.campeoes_lol_corrigido.aggregate([
 {
  $lookup: {
   from: "regioes_lol",
   localField: "region",
   foreignField: "_id",
   as: "info_regiao"
  }
 },
 { $unwind: "$info_regiao" },
 {
  $project: {
   _id: 0,
   nome: "$name",
   regiao: "$region",
   identidade: "$info_regiao.identity",
   clima: "$info_regiao.climate"
  }
```

```
  },
  { $sort: { regiao: 1 } }
])
```

b)

```
db.Pokemon.aggregate([
  {
    $lookup: {
      from: "geracoes_pokemon",
      localField: "generation",
      foreignField: "_id",
      as: "geracao_info"
    }
  },
  { $unwind: "$geracao_info" },
  {
    $project: {
      _id: 0,
      name: 1,
      generation: 1,
      nomeGeracao: "$geracao_info.region",
      ano: "$geracao_info.years",
      lendario: "$geracao_info.new_species"
    }
  },
  { $limit: 10 }
])
```

Exercício 2 - $switch - Classificação com Texto + Quantidade (1,5 pontos) - MÉDIO Contexto: O time de balanceamento quer saber quantos personagens são Fracos, Médios, Fortes, Lendários - mas com texto, não só número!

a) (0,8) Com pokemon: Classifique por attack com $switch: <50 Fraco, <80 Médio, <100 Forte, senão Lendário. Depois $group por classe com total $sum:1 e média de HP. Projete classe, total, mediaHP com $round.

Resultado esperado: { classe: 'Médio', total: 350, mediaHP: 65.2 } - Texto + Quantidade!

b) (0,7) Com campeoes: Classifique por vida com $switch: <550 Frágil, <600 Médio, <650 Tanque, senão Super Tanque. Depois $group por classe com total e média de dano_ataque.

Resultado: { classe: 'Tanque', total: 12, mediaDano: 62.5 }

Cole as 2 queries aqui:

Resposta:

a)

```
db.Pokemon.aggregate([
 {
  $project: {
   niveisDePoder: {
    $switch: {
     branches: [
      { case: { $lt: ["$attack", 50] }, then: "Fraco" },
      { case: { $lt: ["$attack", 80] }, then: "Médio" },
      { case: { $lt: ["$attack", 100] }, then: "Forte" }
     ],
     default: "Lendário"
    }
   },
   hp: 1
  }
 },
 {
  $group: {
   _id: "$niveisDePoder",
   total: { $sum: 1 },
   mediaHP: { $avg: "$hp" }
  }
 },
 {
  $project: {
   _id: 0,
   niveisDePoder: "$_id",
   total: 1,
   mediaHP: { $round: ["$mediaHP", 1] }
  }
 }
])
```

b)

```
db.campeoes_lol_corrigido.aggregate([
 {
```

```
  $project: {
    classe: {
      $switch: {
        branches: [
          { case: { $lt: ["$stats.hp", 550] }, then: "Frágil" },
          { case: { $lt: ["$stats.hp", 600] }, then: "Médio" },
          { case: { $lt: ["$stats.hp", 650] }, then: "Tanque" }
        ],
        default: "Super Tanque"
      }
    },
    dano: "$stats.attack"
  }
 },
 {
  $group: {
    _id: "$classe",
    total: { $sum: 1 },
    mediaDano: { $avg: "$dano" }
  }
 },
 {
  $project: {
    _id: 0,
    classe: "$_id",
    total: 1,
    mediaDano: { $round: ["$mediaDano", 1] }
  }
 }
])
```

Exercício 3 - $bucket + $lookup - Histograma com Texto e Dica Extra (2,0 pontos) - AVANÇADO

Contexto: O diretor pediu histograma, mas com texto bonito e dica para o jogador, não só _id numérico!

a) (1,0) Com pokemon + faixas_ataque: Faça $bucket por attack com boundaries [0,50,80,100,200], default 200, output total $sum:1 e mediaHP $avg. Depois $lookup em faixas_ataque (localField _id, foreignField _id) para buscar classe textual, dica e cor. $unwind e $project com faixa_min, classe, total, mediaHP, dica, cor. Ordene por faixa_min.
Resultado esperado: { classe: 'Médio', total: 350, mediaHP: 65.2, dica: 'Está no caminho!', cor: 'azul' } - Texto + Quantidade + Info extra!

b) (1,0) Com campeoes + faixas_vida: Faça $bucket por estatisticas.vida com boundaries [0,550,600,650,1000], default 1000, output total e nomes com $push. Depois $lookup em faixas_vida para classe e estrategia. Projete classe, total, estrategia e 3 exemplos de nomes com $slice.
Resultado: { classe: 'Tanque', total: 10, estrategia: 'Pode ir na frente', nomes: ['Garen','Darius','Leona'] }

Cole as 2 queries aqui:

Resposta:

a)

```
db.Pokemon.aggregate([
 {
  $bucket: {
   groupBy: "$attack",
   boundaries: [0, 50, 80, 100, 200],
   default: 200,
   output: {
    total: { $sum: 1 },
    mediaHP: { $avg: "$hp" }
   }
  }
 },
 {
  $addFields: {
   id_faixa: {
    $switch: {
     branches: [
      { case: { $eq: ["$_id", 0] }, then: 1 },
      { case: { $eq: ["$_id", 50] }, then: 2 },
      { case: { $eq: ["$_id", 80] }, then: 3 },
      { case: { $eq: ["$_id", 100] }, then: 4 }
     ],
     default: 5
    }
   }
  }
 },
 {
```

```
  $lookup: {
    from: "faixas_ataque_pokemon",
    localField: "id_faixa",
    foreignField: "_id",
    as: "info_faixa"
  }
 },
 { $unwind: "$info_faixa" },
 {
  $project: {
    _id: 0,
    faixa_min: "$_id",
    classe: "$info_faixa.class",
    total: 1,
    mediaHP: { $round: ["$mediaHP", 1] },
    dica: "$info_faixa.tip",
    cor: "$info_faixa.cor"
  }
 },
 { $sort: { faixa_min: 1 } }
])
```

b)

```
db.campeoes_lol_corrigido.aggregate([

 {

  $bucket: {

    groupBy: "$stats.hp",

    boundaries: [0, 550, 600, 650, 1000],

    default: 1000,

    output: {

      total: { $sum: 1 },

      nomes: { $push: "$name" }

    }

  }
```

```
},
{
 $addFields: {
  id_faixa: {
   $switch: {
    branches: [
     { case: { $eq: ["$_id", 0] }, then: 1 },
     { case: { $eq: ["$_id", 550] }, then: 2 },
     { case: { $eq: ["$_id", 600] }, then: 3 },
     { case: { $eq: ["$_id", 650] }, then: 4 }
    ],
    default: 4
   }
  }
 }
},
{
 $lookup: {
  from: "faixas_vida_lol",
  localField: "id_faixa",
  foreignField: "_id",
  as: "info_vida"
 }
},
{ $unwind: "$info_vida" },
```

```
 {
  $project: {
   _id: 0,
   faixa_min: "$_id",
   classe: "$info_vida.classe",
   total: 1,
   estrategia: "$info_vida.description",
   nomes: { $slice: ["$nomes", 3] }
  }
 },
 { $sort: { faixa_min: 1 } }
])
```

Exercício 4 - $bucketAuto - Clustering Automático (2,0 pontos) - AVANÇADO

Contexto: O time de Data Science quer que o Mongo decida sozinho as faixas (clustering) - igual K-means simplificado!

a) (0,7) Com pokemon: Faça $bucketAuto por hp com buckets:3, output total e mediaAtaque. Depois faça com buckets:5. O que muda na distribuição? Explique em comentário na query: quantos por cluster em cada caso?
Resultado 3 buckets: ~266 por cluster, faixas definidas pelo Mongo. 5 buckets: ~160 por cluster.

b) (0,7) Com campeoes: Faça $bucketAuto por estatisticas.vida com buckets:3, output total e mediaDano. Depois com buckets:4.

c) (0,6) Compare $bucket (você define faixas) vs $bucketAuto (Mongo define). Em comentário: Qual tem faixas fixas e quantidades variáveis? Qual tem quantidades equilibradas e faixas variáveis? Quando usar cada um?

Cole as queries e os comentários aqui:

Resposta:

a)
3 buckets:

O MongoDB divide os dados de HP em 3 grupos amplos.
A distribuição fica muito equilibrada, resultando em cerca de 266 a 284 pokémons por cluster (ex: 267, 284 e 249), com os limites (min e max) calculados automaticamente pelo motor.

```
db.Pokemon.aggregate([
 {
  $bucketAuto: {
   groupBy: "$hp",
   buckets: 3,
   output: {
    total: { $sum: 1 },
    mediaAtaque: { $avg: "$attack" }
   }
  }
 }
])
```

5 buckets:
Aumentamos a granularidade. O intervalo de cada balde torna-se mais estreito e a quantidade de pokémons por cluster desce para uma média aproximada de 160 documentos por grupo, permitindo uma análise estatística mais detalhada da densidade dos dados.

```
db.Pokemon.aggregate([
 {
  $bucketAuto: {
   groupBy: "$hp",
   buckets: 5,
   output: {
    total: { $sum: 1 },
    mediaAtaque: { $avg: "$attack" }
   }
  }
 }
])
```

b)
3 buckets:

```
db.campeoes_lol_corrigido.aggregate([
```

```
 {
  $bucketAuto: {
   groupBy: "$stats.hp",
   buckets: 3,
   output: {
    total: { $sum: 1 },
    mediaDano: { $avg: "$stats.attack" }
   }
  }
 }
])
```

4 buckets:

```
 db.campeoes_lol_corrigido.aggregate([
 {
  $bucketAuto: {
   groupBy: "$stats.hp",
   buckets: 4,
   output: {
    total: { $sum: 1 },
    mediaDano: { $avg: "$stats.attack" }
   }
  }
 }
])
```

COMPARAÇÃO: **$bucket - $bucketAuto**

1. **O $bucket.** Nós definimos manualmente os limites (boundaries), o que faz com que algumas faixas fiquem sobrecarregadas de documentos e outras quase vazias.

2. **O $bucketAuto.** O MongoDB analisa a distribuição estatística dos dados e calcula dinamicamente os limites (min/max) para garantir que o número de documentos por balde fique o mais equilibrado possível.

3. Usar o **$bucket:** Quando temos regras de negócio rígidas e pré-determinadas.
(ex: faixas de imposto, escalões etários fixos, notas escolares de 0 a 20).

Usar o $**bucketAuto**: Quando estamos em cenários de exploração de dados/Data Science onde não sabemos os limites ideais e queremos que o algoritmo descubra aglomerados (clusters) automaticamente de forma equilibrada.

Exercício 5 - $facet - Dashboard com Vários Relatórios em 1 Query (2,0 pontos) - DESAFIO

Contexto: O CEO falou: 'Não quero 10 queries, quero 1 query que me dê dashboard completo!' Use $facet!

Com pokemon, em UMA única query com $facet, gere 4 relatórios em paralelo: a) porAtaque: $bucket por attack [0,50,80,100,200] com total
b) porHP: $bucket por hp [0,50,80,255] com total
c) porDefense: $bucket por defense [0,50,80,200] com total
d) mediaGeral: $group com média geral de hp, attack, defense e total geral

Resultado: { porAtaque:[...], porHP:[...], porDefense:[...], mediaGeral:[{mediaHP:..., mediaAttack:..., total:800}] } - 4 relatórios em 1 query!

Cole a query aqui:

Resposta:

```
db.Pokemon.aggregate([
 {
  $facet: {
   porAtaque: [
    {
     $bucket: {
      groupBy: "$attack",
```

```
        boundaries: [0, 50, 80, 100, 200],
        default: 200,
        output: { total: { $sum: 1 } }
      }
    }
  ],
  porHP: [
    {
      $bucket: {
        groupBy: "$hp",
        boundaries: [0, 50, 80, 255],
        default: 255,
        output: { total: { $sum: 1 } }
      }
    }
  ],
  porDefense: [
    {
      $bucket: {
        groupBy: "$defense",
        boundaries: [0, 50, 80, 200],
        default: 200,
        output: { total: { $sum: 1 } }
      }
    }
  ],
  mediaGeral: [
    {
      $group: {
        _id: "estatisticas_gerais",
        mediaHP: { $avg: "$hp" },
        mediaAttack: { $avg: "$attack" },
        mediaDefense: { $avg: "$defense" },
        totalGeral: { $sum: 1 }
      }
    }
  ]
 }
}
])
```

Exercício 6 - DESAFIO FINAL BÔNUS - Tudo Junto: $facet + $bucket + $bucketAuto + $lookup + $switch (1,0 ponto bônus) - NINJA

Contexto: O CEO amou o dashboard e quer o MEGA dashboard: manual vs automático vs classificação textual, tudo em 1 query, com texto, quantidade e dicas! É o relatório que vai para o conselho!

Com pokemon + faixas_ataque, em UMA query com $facet:
a) porAtaqueManual: $bucket por attack [0,50,80,100,200] com total + $lookup em faixas_ataque para classe textual e dica (Texto+Quantidade+Info extra) b) porAtaqueAuto: $bucketAuto por attack buckets:4 com total
c) porHP: $bucket por hp [0,50,80,255] com total
d) classificacaoSwitch: $addFields com $switch para classe textual + $group com total (igual Ex2 mas dentro do $facet)

Resultado: Dashboard completo com manual com lookup (texto+quantidade+dica) + auto + porHP + switch - tudo em 1 query! Juntou lookup→facet→switch→bucket! Se fizer funcionar, ganha 1,0 bônus e título de Ninja MongoDB!

Cole a query aqui:

Resposta:

```
db.Pokemon.aggregate([
 {
  $facet: {
   porAtaqueManual: [
    {
     $bucket: {
      groupBy: "$attack",
      boundaries: [0, 50, 80, 100, 200],
      default: 200,
      output: { total_pokemons: { $sum: 1 } }
     }
    },
    {
     $addFields: {
      id_faixa: {
       $switch: {
        branches: [
         { case: { $eq: ["$_id", 0] }, then: 1 },
         { case: { $eq: ["$_id", 50] }, then: 2 },
```

```
          { case: { $eq: ["$_id", 80] }, then: 3 },
          { case: { $eq: ["$_id", 100] }, then: 4 }
        ],
        default: 5
      }
     }
    }
   },
   {
    $lookup: {
     from: "faixas_ataque_pokemon",
     localField: "id_faixa",
     foreignField: "_id",
     as: "dados_extras"
    }
   },
   { $unwind: "$dados_extras" },
   {
    $project: {
     _id: 0,
     limite_inferior: "$_id",
     nivel_ataque: "$dados_extras.class",
     total_pokemons: 1,
     descricao_estrategia: "$dados_extras.tip"
    }
   },
   { $sort: { limite_inferior: 1 } }
  ],
  porAtaqueAuto: [
   {
    $bucketAuto: {
     groupBy: "$attack",
     buckets: 4,
     output: { total_pokemons: { $sum: 1 } }
    }
   }
  ],
  porHP: [
   {
    $bucket: {
     groupBy: "$hp",
```

```
          boundaries: [0, 50, 80, 255],
          default: 255,
          output: { total_pokemons: { $sum: 1 } }
        }
      }
    ],
    classificacaoSwitch: [
     {
      $addFields: {
       perfil_poder: {
        $switch: {
         branches: [
          { case: { $lt: ["$attack", 50] }, then: "Ataque Baixo" },
          { case: { $lt: ["$attack", 80] }, then: "Ataque Moderado" },
          { case: { $lt: ["$attack", 110] }, then: "Ataque Alto" }
         ],
         default: "Ataque Extremo"
        }
       }
      }
     },
     {
      $group: {
       _id: "$perfil_poder",
       total_pokemons: { $sum: 1 }
      }
     },
     { $sort: { total_pokemons: -1 } }
    ]
   }
  }
]).toArray()
```